

From Nand to Tetris

NPU Extension

# Chapter 3: GEMM 控制器与卷积数据通路

GEMM Controller and Convolution Data Path

Project

Building a Modern Computer From First Principles

# Background

脉动阵列只能做矩阵乘，还不能直接吃图片。本章在阵列外面加“神经系统”：SRAM、DMA/控制器、im2col 或 line buffer、ReLU、MaxPool 和全连接。

阵列的输出格式是“一个空间位置的所有输出通道”，正好适合逐像素扫描的卷积控制器。

# Objective

实现 `n2t\_gemm\_ctrl`（或 `n2t\_conv\_unit`）：把输入特征图切成矩阵乘喂给阵列。

实现激活与池化单元：ReLU（量化版）与 MaxPool 2x2。

实现全连接层：FC 直接复用阵列的矩阵乘能力。

用 Python 生成的小规模 golden 数据完成 RTL 对比验证。

# Deliverables

- `assignment/07/` 下的控制器、激活/池化、全连接模块。
- 对应的手工 testbench 与 golden 数据。
- Makefile 新增 `sim-07` 目标并全部 PASS。

# Contract

- 卷积输出必须与 Python int8 参考实现逐元素一致。
- 支持 stride=1/2、padding=0/1 的配置。
- 数据通路与控制分离：控制器产生地址和使能，数据通路只做运算。
- 保持 posedge 时序约定。

# Resources

- `docs/npu.md` 的路线图。
- `solution/06/SystolicArray.v`：本章复用的计算核心。
- PyTorch / torchvision：用于生成 golden（本机已安装）。

# Tips

- 先从 stride=1、padding=0 的 3x3 卷积开始，跑通后再加 padding。
- 输入通道大于 8 时做通道 tiling：每次取 8 个通道送入阵列。
- 用 FSM 控制状态：IDLE -> LOAD\_W -> RUN -> DRAIN -> DONE。
- golden 先用 Python 整数运算生成，避免浮点误差干扰调试。